

PowerShell Logging And Restriction Rollout Guide

Purpose

This guide describes how to enable PowerShell logging across Windows endpoints, centralize the logs in Splunk or another collector, and use that telemetry to safely move toward PowerShell signing and execution restrictions.

The recommended strategy is:

1. Enable high-value logging first.
2. Forward logs centrally.
3. Build detections and measure normal usage.
4. Remove legacy bypass paths.
5. Pilot script signing and App Control.
6. Enforce restrictions in rings.

Do not start with blocking. Start with visibility.

Recommended Architecture

There are two common collection models.

Option 1: Splunk Universal Forwarder On Endpoints

Use this if the Splunk Universal Forwarder is already deployed broadly.

Each endpoint:

- Generates PowerShell, process creation, and App Control logs.
- Runs the Splunk Universal Forwarder.
- Sends selected Windows Event Logs directly to Splunk indexers or heavy forwarders.

This model is straightforward and gives good endpoint fidelity, but it requires endpoint agent coverage and configuration management.

Option 2: Windows Event Forwarding To Collectors

Use this if you want fewer endpoint-facing Splunk agents.

Each endpoint:

- Generates PowerShell, process creation, and App Control logs.
- Uses Windows Event Forwarding (WEF) to send selected events to Windows Event Collector (WEC) servers.

Each collector:

- Receives forwarded endpoint events in **ForwardedEvents**.
- Runs the Splunk Universal Forwarder.

- Sends **ForwardedEvents** to Splunk.

WEF is useful at scale, but it is passive. It does not enable disabled logs, increase log sizes, change channel permissions, or configure audit policy. Those settings still need to be managed through Group Policy, Intune, or another endpoint configuration tool.

Phase 1: Enable Endpoint Logging

Create a pilot GPO or Intune policy for a small endpoint group first.

PowerShell Logging Policies

For Windows PowerShell 5.1, configure:

```
Computer Configuration
  Administrative Templates
    Windows Components
      Windows PowerShell
```

Enable:

- **Turn on PowerShell Script Block Logging**
- **Turn on Module Logging**
- **Turn on PowerShell Transcription**

For module logging, start with:

```
*
```

This is noisy, but useful during discovery. After the environment is understood, tune it to high-value modules and administrative use cases.

For PowerShell 7, install the PowerShell Core ADMX templates if needed and configure:

```
Computer Configuration
  Administrative Templates
    PowerShell Core
```

Enable the equivalent settings for:

- Script block logging
- Module logging
- Transcription

Script Block Invocation Logging

Script block invocation logging records start and stop events for commands, script blocks, functions, and scripts. Enable it only in a pilot or for high-risk systems at first because it can generate high event volume.

Process Creation Auditing

Enable process creation auditing so that PowerShell launches are visible even when script block logging does not tell the whole story.

Configure:

```
Computer Configuration
  Policies
    Windows Settings
      Security Settings
        Advanced Audit Policy Configuration
          Audit Policies
            Detailed Tracking
              Audit Process Creation = Success
```

Then enable command line capture:

```
Computer Configuration
  Administrative Templates
    System
      Audit Process Creation
        Include command line in process creation events = Enabled
```

This produces Security Event ID 4688 with the process command line.

Increase Event Log Sizes

PowerShell script block logging is verbose. If log sizes are left at small defaults, useful evidence can roll off quickly.

Use Group Policy Preferences, Intune, or endpoint configuration tooling to set larger channel sizes. A practical target for the main PowerShell operational logs is 500 MB where endpoint storage permits.

Example commands for testing:

```
wevtutil sl "Microsoft-Windows-PowerShell/Operational" /e:true /ms:524288000
wevtutil sl "Windows PowerShell" /ms:134217728
wevtutil sl "PowerShellCore/Operational" /e:true /ms:524288000
```

Consider Protected Event Logging

Script block logs can contain sensitive data, including credentials or tokens if scripts mishandle secrets. For broader production rollout, evaluate Protected Event Logging so sensitive event content is encrypted on endpoints and decryptable only at a secure central location.

Phase 2: Forward Logs To Splunk

Direct Endpoint Forwarding

On endpoints with the Splunk Universal Forwarder, deploy an app with an `inputs.conf` similar to:

```
[WinEventLog://Microsoft-Windows-PowerShell/Operational]
disabled = 0
index = win_powershell
renderXml = 1

[WinEventLog://Windows PowerShell]
disabled = 0
index = win_powershell
renderXml = 1

[WinEventLog://PowerShellCore/Operational]
disabled = 0
index = win_powershell
renderXml = 1

[WinEventLog://Security]
disabled = 0
index = wineventlog
renderXml = 1
```

Use the full channel name shown in Event Viewer for non-default logs.

WEF Collector Forwarding

If endpoints forward events to Windows Event Collector servers, configure the Splunk Universal Forwarder on the collectors:

```
[WinEventLog://ForwardedEvents]
disabled = 0
index = wineventlog
renderXml = 1
```

Make sure the WEF subscription includes the relevant PowerShell, Security, and App Control channels.

Event IDs And Channels To Collect

Collect at minimum:

Source	Event IDs	Purpose
Microsoft-Windows-PowerShell/Operational	4104	Script block content
Windows PowerShell	4103, 400, 403, 600	Module logging and engine lifecycle
PowerShellCore/Operational	4104	PowerShell 7 script block content
Security	4688	Process creation with command line
Security	1102	Security log cleared
AppLocker or Code Integrity channels	Policy dependent	App Control, CLM, and allow or block telemetry

Also collect Windows Event Forwarding health events from collectors so failed subscriptions are visible.

Phase 3: Build Detections

Start with detections that are easy to act on.

Suspicious PowerShell Command Lines

Alert on 4688 where **New Process Name** or command line includes:

- powershell.exe
- pwsh.exe
- -EncodedCommand
- -enc
- -ExecutionPolicy Bypass
- -NoProfile
- -WindowState Hidden
- IEX
- Invoke-Expression
- DownloadString
- FromBase64String
- System.Net.WebClient
- Invoke-WebRequest
- Invoke-RestMethod

Suspicious Parent Processes

Alert when PowerShell is spawned by:

- Microsoft Office applications
- Browsers
- Teams
- OneDrive
- wscript.exe

- `cscript.exe`
- `mshta.exe`
- `rundll32.exe`
- `regsvr32.exe`
- Archive tools
- PDF readers

Suspicious Paths

Alert when PowerShell, scripts, or modules execute from user-writable locations:

- `%TEMP%`
- `%APPDATA%`
- `%LOCALAPPDATA%`
- Downloads
- Desktop
- Recycle Bin
- Browser cache paths
- Unusual UNC paths

Logging And Policy Tampering

Alert on:

- Security log clear: `1102`
- PowerShell log clear events
- Changes to PowerShell policy registry keys
- Disabling script block logging
- Disabling transcription
- Changes to App Control policy state
- PowerShell launched with `-ExecutionPolicy Bypass`

Administrative Blast Radius

For SharePoint and Microsoft 365 administration, monitor:

- Permission changes
- Site collection permission changes
- Sharing policy changes
- Entra application permission grants
- Use of broad Microsoft Graph or SharePoint permissions such as tenant-wide full control grants

For sensitive automation, prefer scoped automation identities and Just Enough Administration (JEA) over broad interactive PowerShell.

Phase 4: Remove Legacy Bypass Paths

Remove PowerShell 2.0

PowerShell 2.0 is a legacy downgrade path and should be removed unless a documented business requirement exists.

Pilot first, then remove using endpoint management tooling.

Example command for local testing from an elevated PowerShell prompt:

```
Disable-WindowsOptionalFeature -Online -FeatureName  
MicrosoftWindowsPowerShellV2Root
```

Verify with:

```
Get-WindowsOptionalFeature -Online -FeatureName *PowerShellV2*
```

Phase 5: Move Toward Restriction

Execution Policy Is Not Enough

Execution policy can be useful as a guardrail, but it is not a security boundary. Users can bypass it in multiple ways, including pasting commands directly into an interactive session or launching a process with a less restrictive process-level policy.

Use execution policy to support signing hygiene, not as the final enforcement layer.

Introduce Signing

Use **AllSigned** for approved automation once script ownership and signing workflows exist.

Recommended prerequisites:

- A code-signing certificate lifecycle.
- A signing process in CI/CD or a controlled admin workstation.
- Source control for production scripts.
- Named human owner for each script.
- Peer review for scripts that run unattended or modify permissions at scale.
- A revocation and emergency exception process.

Example GPO path:

```
Computer Configuration  
  Administrative Templates  
    Windows Components  
      Windows PowerShell  
        Turn on Script Execution  
          Allow only signed scripts
```

Use App Control For Real Enforcement

Use App Control for Business, formerly Windows Defender Application Control (WDAC), for stronger control.

Recommended path:

1. Create an App Control policy in audit mode.
2. Pilot on IT and representative power-user machines.
3. Review script, module, and binary allow events.
4. Fix unsigned or unmanaged automation.
5. Move standard-user endpoints to enforcement.
6. Keep a documented break-glass process.

App Control can place unapproved interactive PowerShell into Constrained Language Mode (CLM). This is the preferred path for limiting what interactive users can do while still allowing approved signed automation to run with the required language capabilities.

Rollout Plan

Ring 0: Lab

Validate:

- GPO or Intune policy applies.
- Logs are generated.
- Logs are readable locally.
- Splunk or WEF collection works.
- Event volume is understood.
- No critical workflows break.

Ring 1: IT Pilot

Include:

- Security team endpoints.
- Endpoint engineering.
- Help desk representatives.
- SharePoint or Microsoft 365 administrators.

Goals:

- Find legitimate administrative patterns.
- Tune logging volume.
- Build allowlists.
- Identify scripts that need owners and signatures.

Ring 2: Power Users

Include users who already run scripts or admin tools.

Goals:

- Validate normal business usage.
- Identify unmanaged scripts.
- Move recurring scripts into source control.
- Start signing production scripts.

Ring 3: Broad Endpoint Logging

Enable logging broadly once:

- Splunk capacity is sized.
- Detections are tuned.
- False positives are manageable.
- Endpoint log sizes are adequate.

Ring 4: Restriction

Apply restrictions gradually:

- Remove PowerShell 2.0.
- Enforce signed scripts for managed automation.
- Apply App Control in audit mode.
- Enforce App Control for standard users.
- Expand CLM/App Control enforcement to higher-risk populations.

Operational Practices

Maintain:

- A documented script owner for every recurring script.
- A script inventory.
- A signing standard.
- A break-glass process.
- A process for expiring or transferring ownership when users change roles.
- A review cadence for Splunk detections and App Control policy events.

Treat scripts that other people depend on, that run unattended, or that modify permissions at scale as production software. They should follow the SDLC, including source control, review, testing, and signing.

Quick Validation Commands

Check effective execution policy:

```
Get-ExecutionPolicy -List
```

Check for recent script block events:

```
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -MaxEvents 20 |  
Where-Object Id -eq 4104 |
```

```
Select-Object TimeCreated, Id, ProviderName, Message
```

Check process creation events:

```
Get-WinEvent -FilterHashtable @{LogName = "Security"; Id = 4688; StartTime = (Get-Date).AddHours(-1)} -MaxEvents 20 |  
    Select-Object TimeCreated, Id, ProviderName, Message
```

Check PowerShell 2.0 optional feature state:

```
Get-WindowsOptionalFeature -Online -FeatureName *PowerShellV2*
```

References

- Microsoft PowerShell Group Policy settings: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_group_policy_settings
- Microsoft PowerShell logging on Windows: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows
- Microsoft Windows PowerShell 5.1 logging: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging
- Microsoft execution policy guidance: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies
- Microsoft App Control script enforcement: <https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/script-enforcement>
- Microsoft PowerShell security features: <https://learn.microsoft.com/en-us/powershell/scripting/security/security-features>
- Microsoft Windows Event Forwarding guidance: <https://learn.microsoft.com/en-us/windows/security/operating-system-security/device-management/use-windows-event-forwarding-to-assist-in-intrusion-detection>
- Microsoft command line process auditing: <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/component-updates/command-line-process-auditing>
- Splunk Windows Event Log monitoring: <https://help.splunk.com/en/splunk-enterprise/get-started/get-data-in/9.3/get-windows-data/monitor-windows-event-log-data-with-splunk-enterprise>
- Splunk inputs.conf reference: <https://help.splunk.com/en/splunk-enterprise/administer/admin-manual/9.2/configuration-file-reference/9.2.0-configuration-file-reference/inputs.conf>